🛡️ Module 11: Reverse Engineering, Fuzzing and Binary Exploitation

```
# Exercise 02: Performing Binary Analysis on a 64-bit Linux
```

Scenario

Binary Analysis on a 64-bit Linux System focuses on understanding the architectural extensions of x86-64 (AMD64) ELF binaries. Compared to 32-bit (x86) architectures, 64-bit Linux introduces 64-bit general-purpose registers (`RAX`, `RBX`, `RCX`, `RDX`, `RSI`, `RDI`, `RBP`, `RSP`, and `R8` through `R15`), updated 64-bit System V AMD64 ABI calling conventions, and distinct system call numbers (`sys_write` at syscall **1**, `sys_exit` at syscall **60** defined in `/usr/include/x86_64-linux-gnu/asm/unistd_64.h`).

In this exercise, we log into the **Software-Test-Linux** (64-bit) machine (**student** / **studentpassword**), inspect 64-bit CPU registers using **GDB** (**gdb -q /bin/bash** -> **break main** -> **run** -> **info registers**), verify system CPU architecture via **lscpu**, inspect system call header files (**unistd_64.h**), utilize Python 2.7 (**2.7.18**) string manipulation methods (`c[::-1].encode('hex')`) to reverse and convert string payloads into Little-Endian hexadecimal byte format for assembly shellcode construction, and perform full ELF structure and symbol table inspection on a 64-bit sample executable (**helloworld64-s**) using **readelf** (**readelf -a -W ~/examples/samplecode/helloworld64-s** and **readelf -s ~/examples/samplecode/helloworld64-s**).

Objective

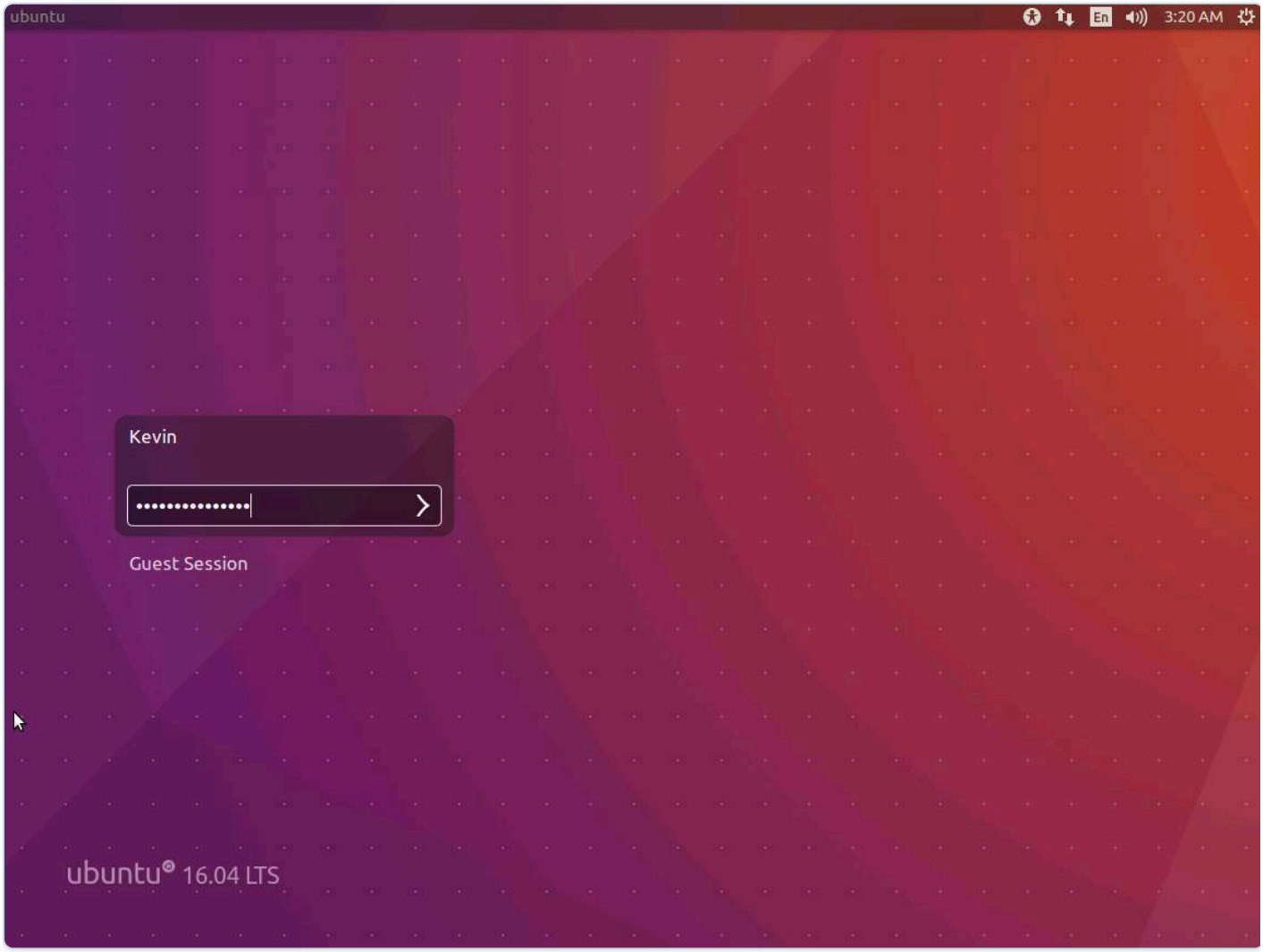
The objective of this lab is to help students learn how to:
- Authenticate to 64-bit Linux test environment (**student** / **studentpassword**)
- Inspect 64-bit processor registers (`RAX`, `RBX`, `RCX`, `RDX`, `RSI`, `RDI`, `RBP`, `RSP`, `R8` - `R15`) in GDB via **info registers**
- Inspect system architecture details using **lscpu**
- Examine 64-bit x86 Linux system call numbers via **cat /usr/include/x86_64-linux-gnu/asm/unistd_64.h**
- Leverage Python 2.7.18 shell to reverse string endianness and convert ASCII strings to hex bytes (**c[::-1].encode('hex')**) for shellcode development
- Perform full ELF binary header, program header, and section analysis on 64-bit binary **helloworld64-s** using **readelf -a -W ~/examples/samplecode/helloworld64-s**
- Inspect 64-bit binary symbol tables (`.symtab`) and dynamically linked functions (`printf`) using **readelf -s ~/examples/samplecode/helloworld64-s**
- Determine Python version installed on target system: **2.7.18**

Task 1: Perform 64-bit Binary Analysis, Register Inspection, and Python Shellcode Preprocessing

In this task, we log into **Software-Test-Linux** (64-bit), inspect 64-bit CPU registers using GDB, review 64-bit system calls (`unistd_64.h`), use Python 2.7.18 to format string payloads into hex byte arrays, and inspect 64-bit ELF headers and symbol tables via `readelf`.

Lab Tasks

1. Log into **Software-Test-Linux** 64-bit VM using credentials (**student** / **studentpassword**).



1. Launch GDB on `/bin/bash`, set a breakpoint at main, run execution, and display 64-bit registers:

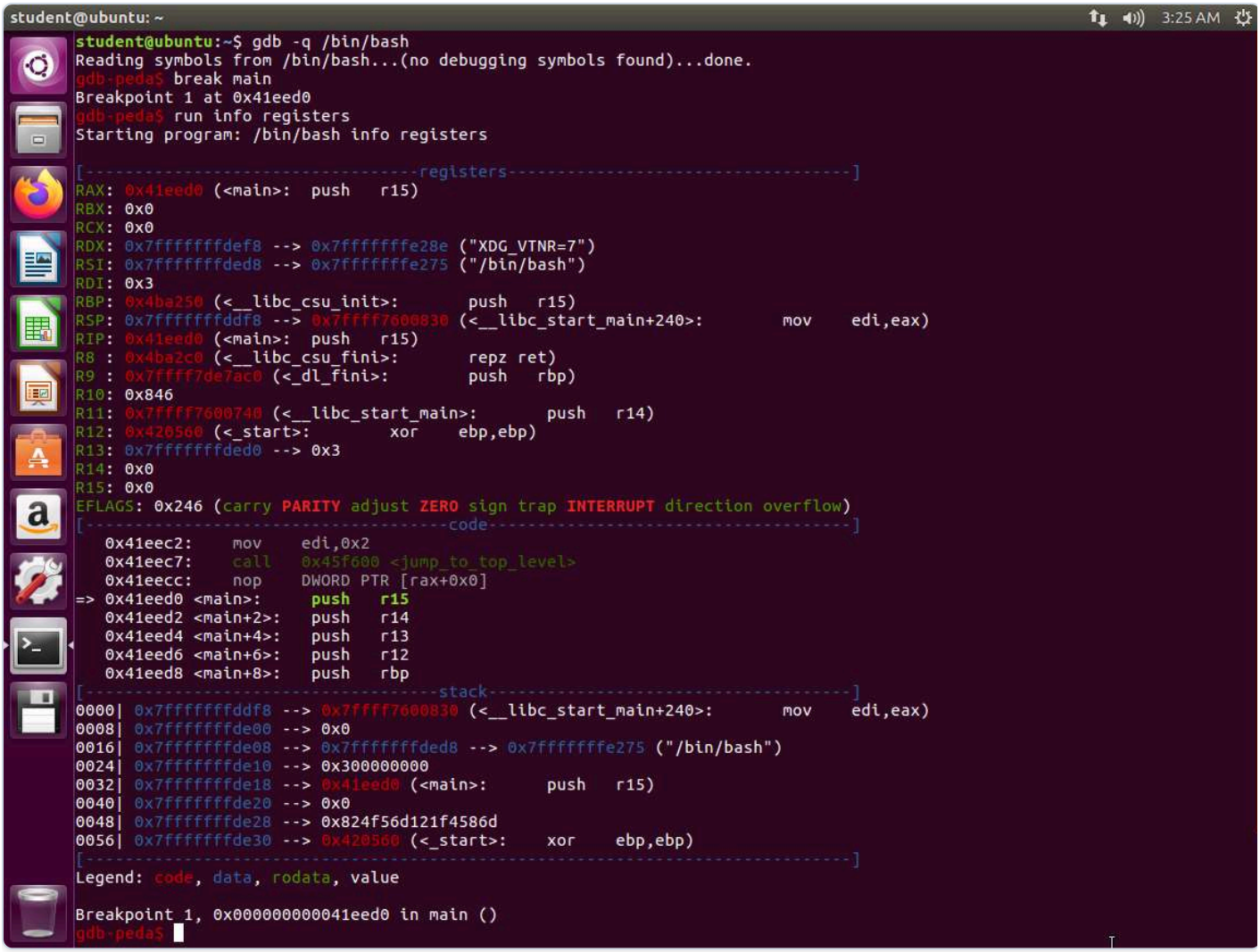
   gdb -q /bin/bash

   break main

   run

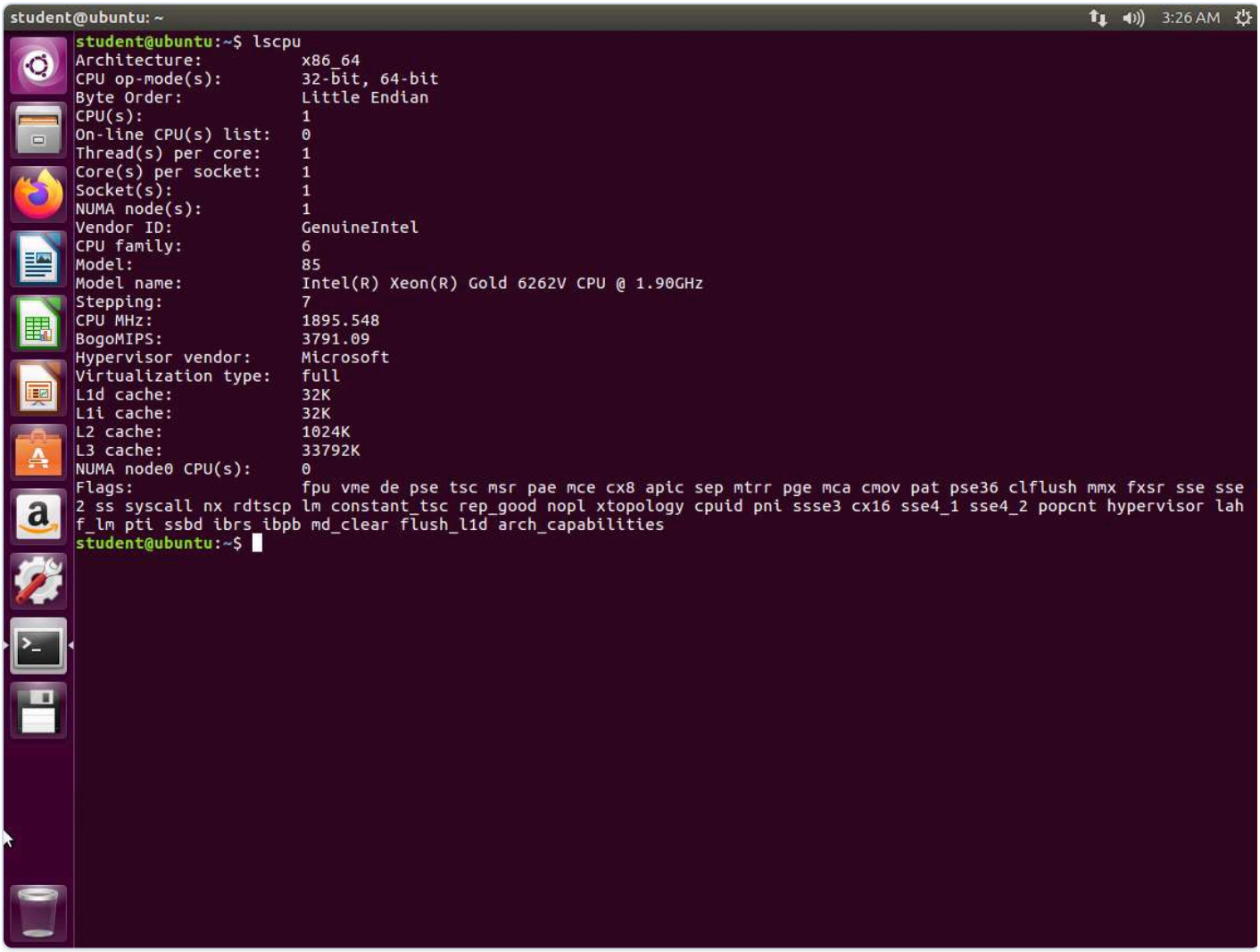
   info registers

   Observe 64-bit extended registers (RAX, RBX, RCX, RDX, RSI, RDI, RBP, RSP, R8-R15).

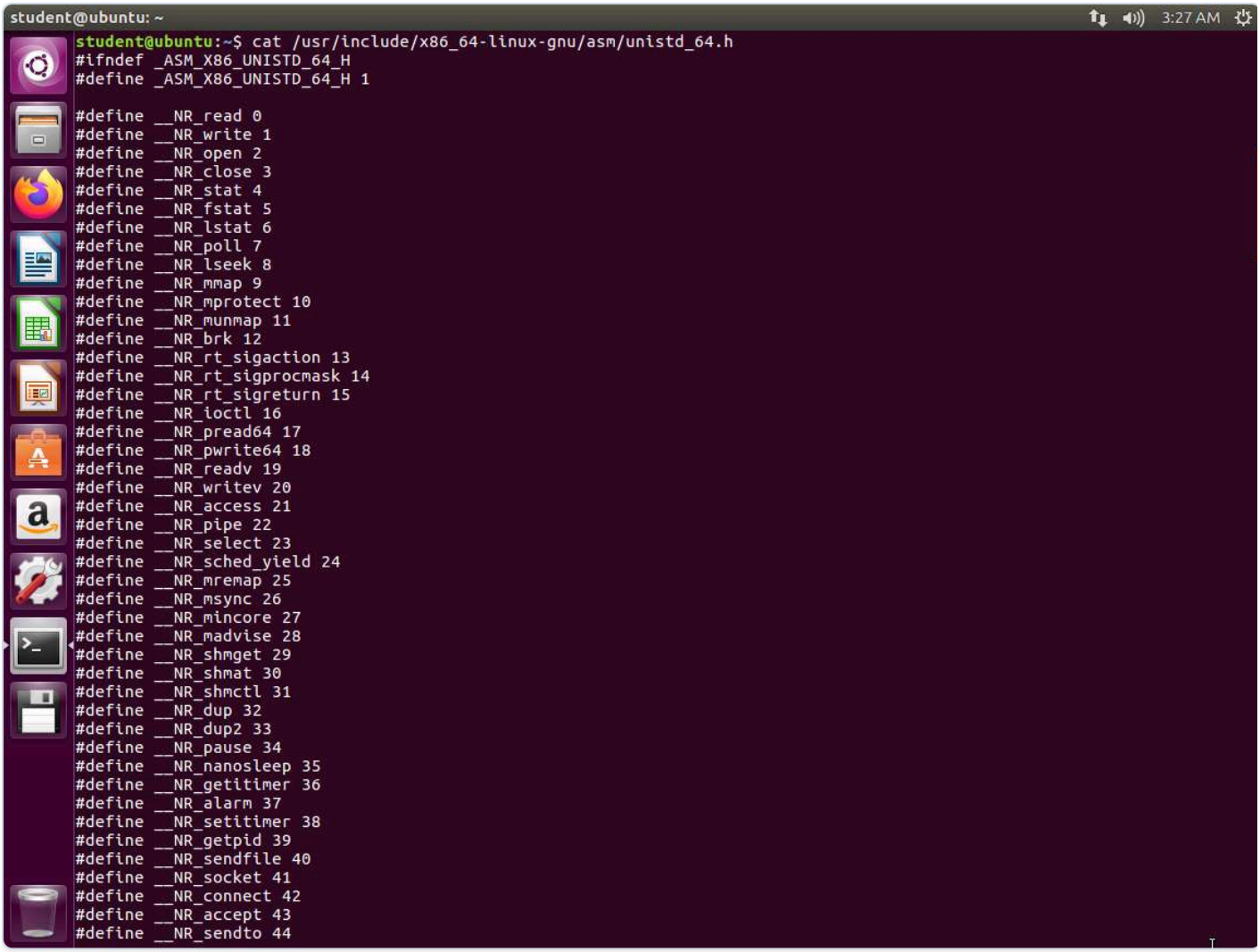


1. Run `lscpu` to verify system CPU architecture details:

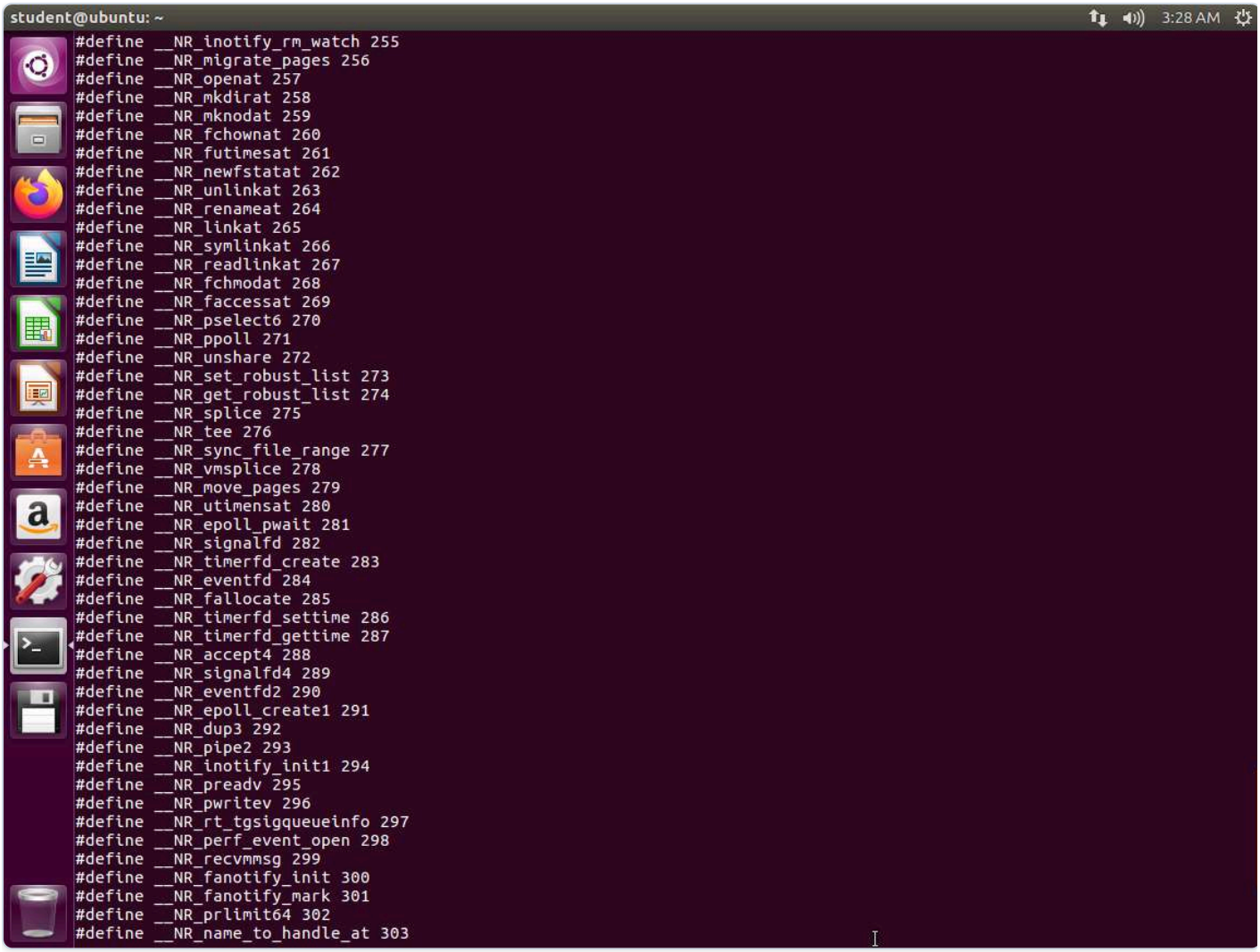
   lscpu



1. Inspect 64-bit Linux system call header file:
   cat /usr/include/x86_64-linux-gnu/asm/unistd_64.h



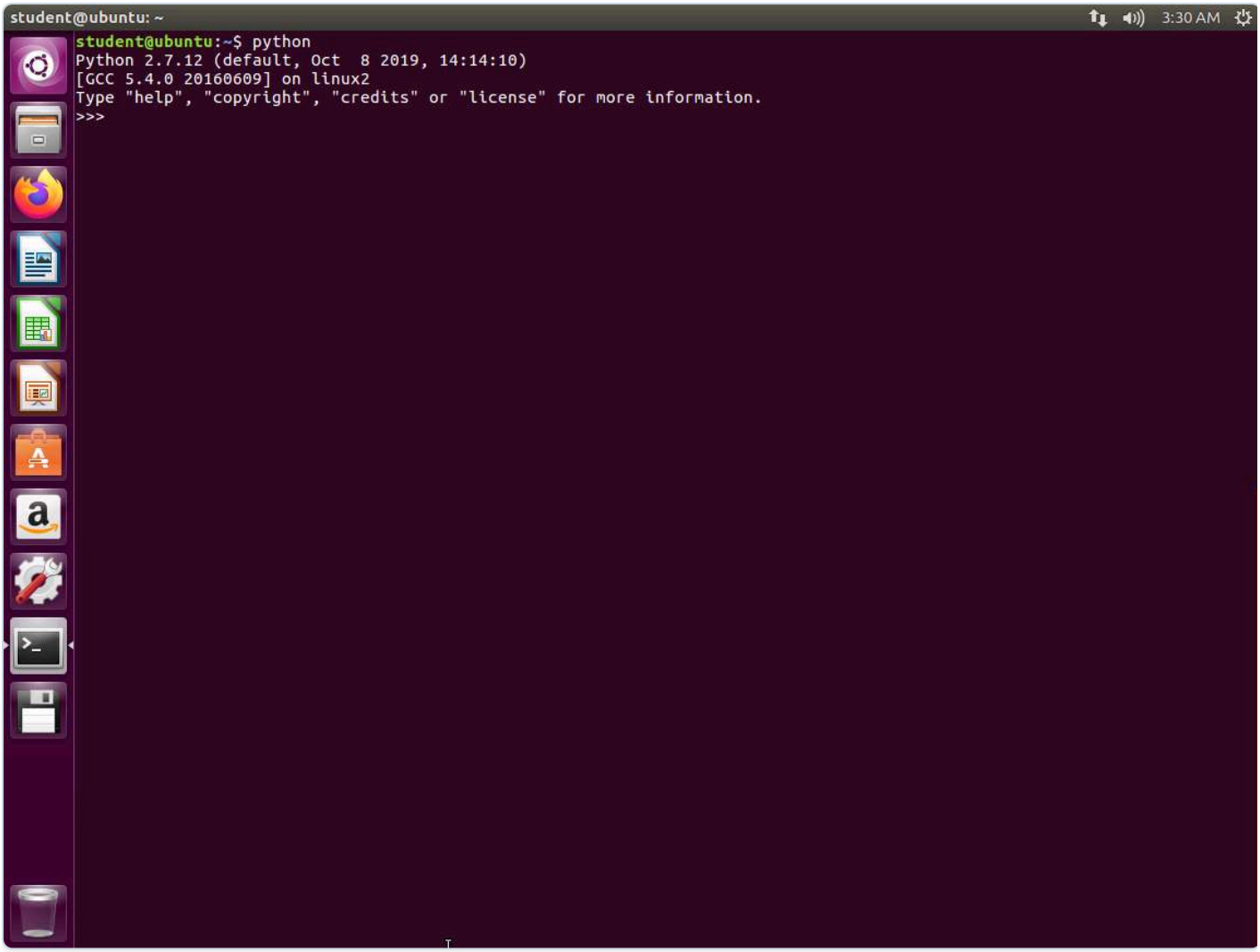
1. Scroll down to review system call table mappings:
   Note: `sys_write` *is syscall ID* **1**, `sys_exit` *is syscall ID* **60**.



1. Launch Python interactive interpreter prompt:

   python

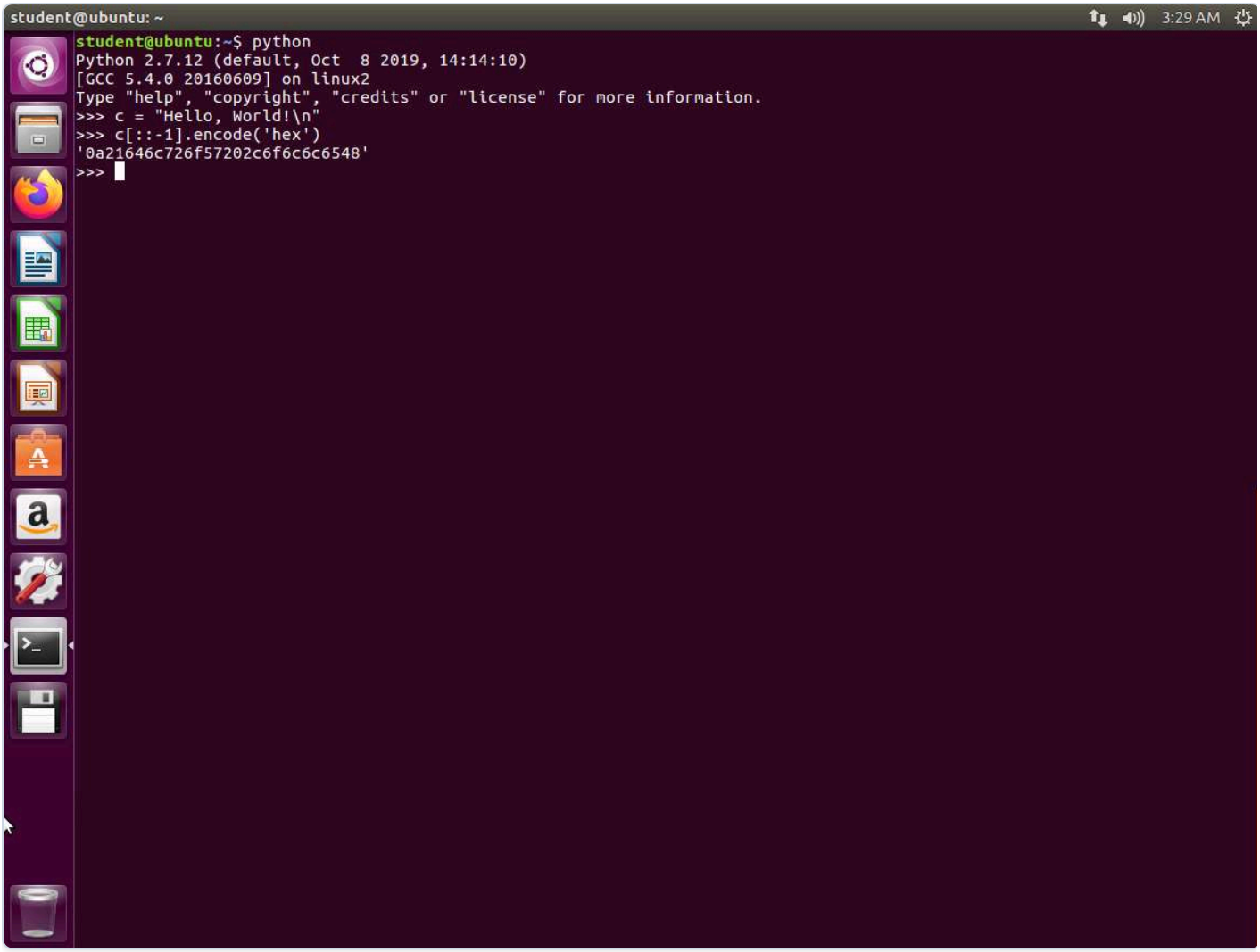
   Note: Python version is **2.7.18**.



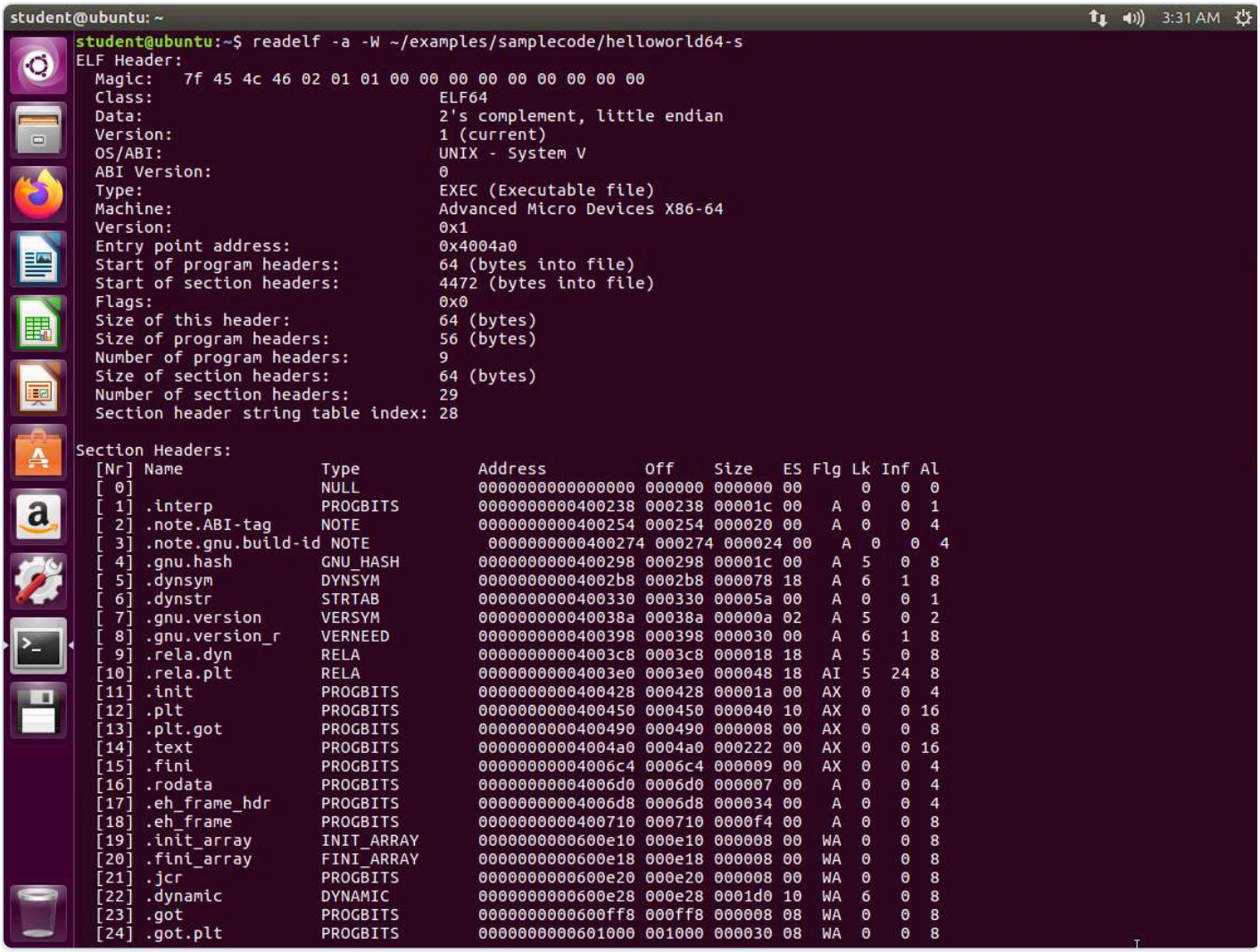
1. Perform string reversal and Little-Endian hex encoding:

   c = "Hello, World!\n"

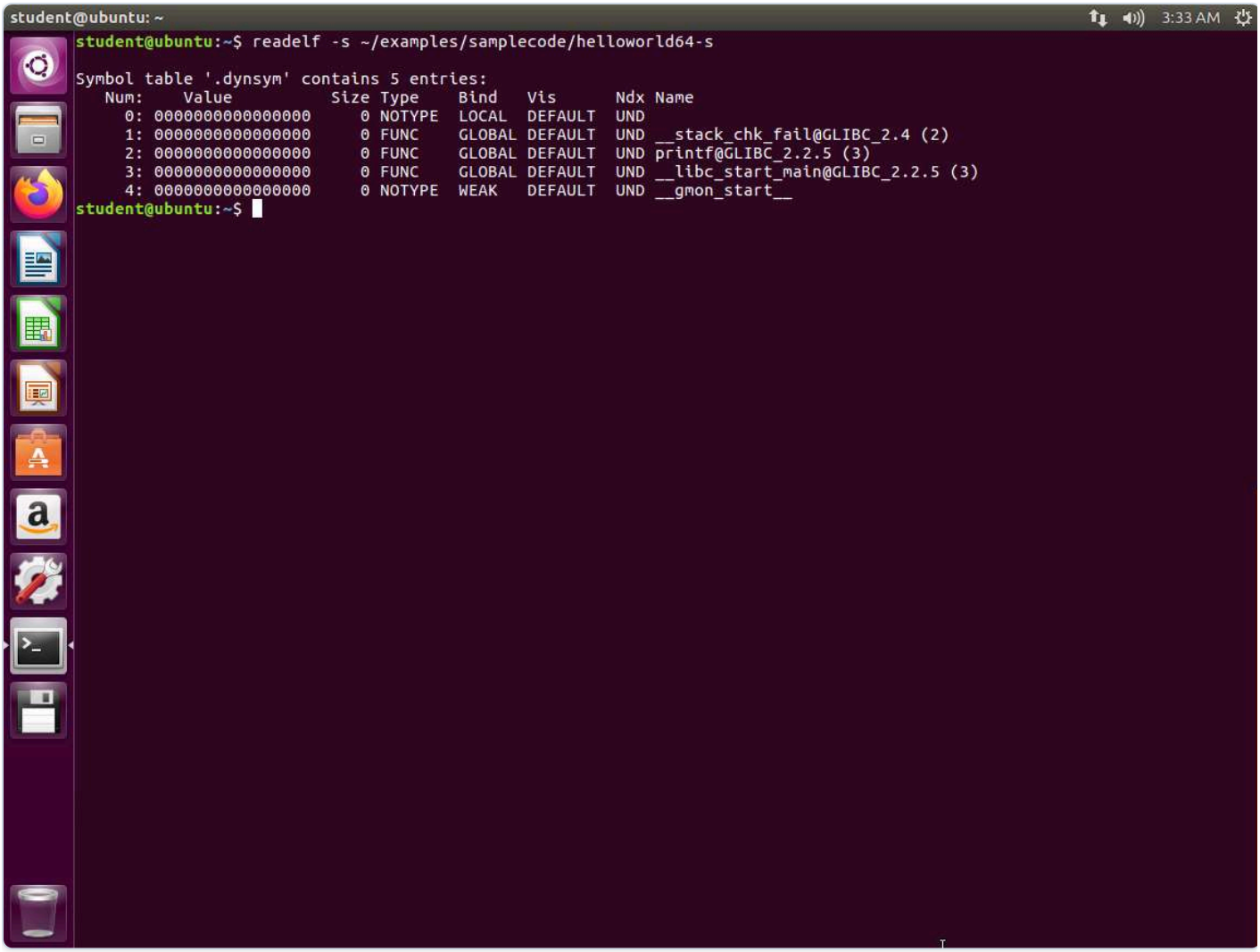
   c[::-1].encode('hex')



1. Perform full wide-format ELF header and section analysis on 64-bit sample executable `helloworld64-s`:
   readelf -a -W ~/examples/samplecode/helloworld64-s



1. Inspect symbol table entries (`.symtab`) for dynamic library functions (`printf`):
   readelf -s ~/examples/samplecode/helloworld64-s



This concludes the lab exercise.

> **IMPORTANT:** ❓ **Question 11.2.1.1** Conduct a binary analysis on the 64-bit Software-Test-Linux system. Utilize Python for string manipulation and determine the Python version installed on the Software-Test-Linux machine. [!SUCCESS] ✅ **Answer:** **2.7.18**